

OS Practical 7

Banker's Algorithm

```
Asus@LAPTOP-M02HFD9C MINGW64 ~/Desktop
$ nano banker.c

Asus@LAPTOP-M02HFD9C MINGW64 ~/Desktop
$ gcc banker.c -o banker

Asus@LAPTOP-M02HFD9C MINGW64 ~/Desktop
$ ./banker
System is in safe state
Safe sequence: P1 P2 P3 P4 P0
```

Code:

```
GNU nano 8.7
#include <stdio.h>

int main() {
    int n = 5; // number of processes
    int m = 4; // number of resources

    int allocation[5][4] = {
        {0,0,1,4},
        {0,6,3,2},
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4}
    };

    int max[5][4] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[4] = {1,5,2,0};

    int need[5][4];
    int finished[5] = {0};
    int safeSeq[5];
    int count = 0;

    // Calculate Need Matrix
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    while(count < n){
        int found = 0;

        for(int i=0;i<n;i++){
            if(finished[i] == 0){
                int j;
                for(j=0;j<m;j++){
                    if(need[i][j] > available[j])
                        break;
                }

                if(j == m){
                    for(int k=0;k<m;k++){
                        available[k] += allocation[i][k];
                    }

                    safeSeq[count++] = i;
                    finished[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0){
            printf("System is not in safe state");
            return 0;
        }

        printf("System is in safe state\nSafe sequence: ");

        for(int i=0;i<n;i++){
            printf("P%d ", safeSeq[i]);
        }

        return 0;
    }
}
```